Scientific Reports

https://doi.org/10.1038/s41598-026-52626-0

Article in Press

An adaptive AI-enabled framework for cognitive data management and real-time optimization of industrial task processing

Received: 25 March 2026

Accepted: 6 May 2026

Published online: 13 May 2026

Cite this article as: Zhang D., Wang Z., Shi T. *et al.* An adaptive AI-enabled framework for cognitive data management and real-time optimization of industrial task processing. *Sci Rep* (2026). https://doi.org/10.1038/s41598-026-52626-0

Dongmei Zhang, Zongshan Wang, Ting Shi, Shijin Li, D. Akila, S. Baskar & Mohamed Shakeel pethuraj

An Adaptive AI-Enabled Framework for Cognitive Data Management and Real-Time Optimization of Industrial Task Processing

Dongmei Zhang[1], Zongshan Wang[2], Ting Shi[3], Shijin Li[4,5,*], D. AKILA[6], S.Baskar[7], and Mohamed Shakeel pethuraj[8]

[1]College of Electronic Information and Communication Engineering, Chongqing Aerospace Polytechnic, Chongqing, China

[2]R&D Center, Xuchang Tobacco Machinery Company Ltd., Xuchang, China

[3]School of Information Engineering, Yunnan Vocational College of Mechanical and Electrical Technology, Kunming, China

[4]Academic Affairs Office, Yunnan University of Finance and Economics, Kunming, China

[5]Faculty of Management and Economics, Kunming University of Technology, Kunming, China

[6]Department of Computer Applications, Saveetha College of Liberal Arts and Sciences, Saveetha Institute of Medical and Technical Sciences, Chennai, India.

[7]Department of Electronics and Communication Engineering/Centre for Interdisciplinary Research, Karpagam Academy of Higher Education, Coimbatore, India.

[8]Faculty of Computer and Engineering, Universitas Alma Ata, Brawijaya, Yogyakarta, Indonesia.

[*]Corresponding Authors: Shijin Li (shijin_li@hotmail.com)

ABSTRACT

The high-performance industrial processes under dynamic and complicated workloads require efficient allocation of tasks and control of information. Traditional frameworks are based on unrestricted scheduling and have low scalability, low adaptability, and low resource demand, which in turn causes slow information latency and maintenance downtime, eventually reducing operation efficiency and productivity. The proposed research is the Integrated Information Analytical Framework (IIAF) to optimize the management of cognitive data and processing industrial tasks. The framework also includes an initial task category determination facility to allow task-oriented data processing and a smooth transition to the task allocation stream to enhance processing speed and the accuracy of classification. It also uses dynamic scheduling and smart data management in order to minimize information latency and responsiveness to dynamic workloads. There are real-time cognitive decision-making algorithms that are used to optimize the allocation of resources as well as enhance the scalability and throughput of systems. The experiments prove that IIAF enhances allocation rate by 7.03%, accuracy by 11.78%, lowers information latency by 12.03%, and allocates time by 20.89%. Moreover, it can minimize overload tasks by 18.65% and minimize downtime by up to 37.4%, which proves to be efficient in optimizing the industry.

Key Terms—Classifier Learning, integrated information-analytical framework, task identification, Data Analytics, Task Allocation, data management.

1. Introduction

Industrial data processing involves integrating data management and optimizing allocation operations to enhance the quality of specific products [1]. Then, the task is allocated to a specific machine to automate daily routines, streamline repetitive processes, and facilitate informed decision-making [2]. The idle machine data, with its indicators, analyzes the task and monitors its completion, and the industrial sector controls the current status of neighboring machines and provides the

outcomes [3]. Data processing is a rapidly evolving technology that facilitates improvements and profitability in industrial applications [4]. The industrial and the user establish the communication phase to deliver the appropriate material. Industrial data processing includes three analysis stages: administrative, networking, and manufacturing [5].

Recent advances in adaptive AI have demonstrated that continuous learning from real-time sensor streams can significantly enhance responsiveness, fault tolerance, and decision accuracy in dynamic industrial environments [6]. Recent developments in AI-driven scheduling frameworks integrate constraint-aware optimization with intelligent decision systems, which can substantially improve service efficiency, resource utilization, and responsiveness in complex industrial production environments [7]. The task is assigned to the machine, estimating the delay and failure in the processing. It outlines the allocation process and ensures timely computation by making informed decisions [8]. The recommendation is performed as part of the classification process, including the National Industrial Classification (NIC). Based on industry standards, classification is carried out to facilitate the completion process. Here, the traditional product is developed in conjunction with market analysis [9, 10].

Industrial data processing involves the dynamic management of task availability and load-balancing trends to effectively allocate resources and balance the network according to industrial requirements [11]. The training data is used to evaluate the appropriate data processing in the industrial environment. In this case, the structural sequence step defines the data processing by deploying a machine-learning approach [12]. The feature space defines the predictive analysis step in this industrial data processing. The analysis is based on the data volume and velocity and determines the standardized processing [13]. Finally, the task is allocated to the relevant machine, the working condition is stated, and the information allocation is evaluated [14]. This proposal is related to cognitive utilization, functional optimization, and reducing the impact of information lag, thereby decreasing allocation time and enhancing the allocation rate [15]. The IIAF framework is specifically designed for structured industrial data, including machine state parameters, task attributes, and allocation logs. It does not process unstructured data such as free-text maintenance records or images. Specifically, this work proposes the IIAF, which employs a 3-layer feedforward neural network as the pre-classifier for task categorization into priority classes. A cognitive information detection function computes a weighted detection score D from machine-state variables to trigger dynamic reallocation when workload imbalance is detected. In addition, a definitive allocation module uses forwarding weights f_w and confidence parameter c_p to assign tasks to idle machines with minimal delay and balanced utilization. At the supervisory level, a management feedback loop continuously monitors allocation rate, information lag, and downtime in real time through 30–50 second control cycles.

2. Related Works

Kumar et al. [16] proposed a Cognitive-driven analytical model for classifying data using three methodologies: Conventional Neural Networks (CNN), Decision Trees, and Swarm Intelligence. This work aims to improve the accuracy rate and define the features of the data by introducing the Wolf-search algorithm. Here, the deep swarm-optimized classifier is employed to extract the features from the datasets. Algarni et al. [17] introduced the Service-Intensive Calculation Offloading Framework (SIC-OF), which reduces pipelined resource sharing-induced service lag

and calculation delays to improve the quality of edge-cloud computing services. SICOF optimizes request processing and service distribution by recognizing service-free edge and cloud devices and offloading computations and tasks using support vector classification.

Almusawi and Pugazhenthi [18] developed a resource distribution method using honey bee optimization, which employs scout and forager agents to allocate resources dynamically, thereby improving supply chain scheduling like how bees forage. However, the method encounters challenges with real-time adaptability in highly unpredictable demand situations, necessitating further improvements in decision-making speed. In [19], Yuan et al. introduced a feature learning process for improving industrial data processing. Nonlinear and temporal data analytics reduce prediction errors when handling multiple tasks.

The Emergency Task Scheduling Pre-Classification (ETSPC) technique, as presented by Qu et al. [20], optimizes industrial edge network emergency offloading by prioritizing high-concurrency activities through task path analysis and multiple priority levels. An interactive visualization system has been developed to improve the quality-related faults in the time distribution presented by Zhang et al. [21]. In heavy plate production, analysis is used to define data processing in the industry. The data is distributed in a heterogeneous environment to address quality-related issues

Subramani et al. [22] introduced preemptive classification using discrete data models to transmit real-time batch processing for identifying productivity in the agriculture industry. The random forest model examines the history of data matching and probabilistic data substitution. The research results are validated with classification factor and detection accuracy, and though they have benefits, they still suffer from information lag factors. Zhang et al. [23] presented a derivative-free optimization (DFO) method for classifying datasets to achieve reliable optimization. In this work, prediction is carried out by deploying the industrial datasets and illustrating the standardized solution.

Zhang et al. [24] proposed two methodologies: Industrial Big Data Application (I-BD) and reference model. The developed work focuses on the Industry (I) dimension, the Application scenario (A) dimension, and the common service platform (P) dimension. The multi-party co-construction is introduced for the reference model architecture. Intelligent management establishes a unified service platform for large-scale industrial data. Helu et al. [25] introduced a scalable pipeline for analytical applications and derived a feasible solution. Data distribution determines the production network and provides industrial growth.

Here, a multivariate location and scatter matrix are used to minimize case and cell outliers. Then, the second method, the GRE, is used to handle the missing value and detect faulty data by improving the effectiveness of Luo et al. [26]. Zangeneh et al. [27] presented a Uniform Project Ontology for logical deduction and inference in data management. Data utilization, linked data, and the semantic web define the cost normalization process. The project analysis determines the project's knowledge representation.

Zhang et al. [28] utilize mapping with sequence data to identify dynamic features by introducing a deep network and enhancing generalization. Regression is used for data extraction in big data analytics. New unsupervised learning concerning Gaussian-based Dynamic Probabilistic Clustering (GDPC) based on IoT applications was developed by Diaz-Rozo et al. [29]. The model parameter examines the industry's dynamic data processing scenario. Expectation-Maximization (EM) is

designed to provide the membership for less computation. The robustness of data classification is improved; this work aims to enhance accuracy and specificity.

Cirera et al. [30] implemented a data-driven performance evaluation to address industry-related issues. The robust result is enhanced by introducing the Multivariate Kernel Density Estimation technique and Self-Organizing Maps. The abnormal situation is detected to ensure trustworthy data processing and improve performance. Ragazou et al. [31] analyzed the operational efficiencies of various machines in an industry by optimizing data inputs to produce more valuable outputs and enhancing production efficiency through improved performance.

Table 1a. Summary of Existing Works

Reference	Key Concept	Results Achieved	Limitations
Kumar et al. [16]	Cognitive-driven classification using CNN, Decision Trees, & Swarm Intelligence	Improved accuracy & feature classification	Scalability issues in real-time scenarios
Algarni et al. [17]	Service-Intensive Offloading Framework (SIC-OF) for edge-cloud computing	Reduced service lag, improved resource efficiency	Limited real-time adaptability & scalability
Almusawi & Pugazhenthi [18]	Honey Bee Optimization for Resource Distribution	31.4% scalability improvement, 23.7% operational cost reduction	Struggles with real-time adaptability in unpredictable demand scenarios
Yuan et al. [19]	Feature learning process for industrial data processing	Reduced prediction error	No mention of adaptability in dynamic environments
Qu et al., [20]	Emergency Task Scheduling Pre-Classification (ETSPC) for industrial edge networks	19.2% reduction in offloading delay, 45.3% improvement in job completion	Lacks real-world validation, limited in adversarial scenarios
Zhang et al., [21]	An interactive visualization system for quality fault detection	Improved system efficacy, better quality fault identification	Needs alternative algorithm improvements
Subramani et al. [22]	Preemptive classification in the agriculture industry using Random Forest	Improved classification factor & detection accuracy	Suffers from information lag
Zhang et al. [23]	Derivative-Free Optimization (DFO) for dataset classification	Optimized classification for industrial datasets	Imbalance in data processing

Zhang et al. [24]	Industrial Big Data (I-BD) application & reference model	Intelligent data management for big industrial data	Needs better integration in diverse industrial applications
Helu et al. [25]	Scalable pipeline for industrial data analytics	Enhanced scalability in Industrial IoT data processing	Adaptability Issues
Luo et al. [26]	Cellwise Outlier Filters (COF-GRE) for data quality improvement	Improved robustness in data classification	Requires better handling of complex missing data
Zangeneh et al. [27]	Uniform Project Ontology for logical deduction & inference	Enhanced project risk assessment & knowledge transfer	Limited application to broader industrial datasets
Zhang et al. [28]	Automatic dynamic feature extraction for an ensemble tree model	Improved feature detection & classification	Requires more profound generalization in complex scenarios
Dias-Rozo et al. [29]	Gaussian-based Dynamic Probabilistic Clustering (GDPC) for IoT	Improved accuracy & specificity in IoT applications	Needs optimization for real-time processing
Cirera et al. [30]	Multivariate Kernel Density Estimation & Self-Organizing Maps	Trustworthy data processing & enhanced performance	Lacks abnormal system handling
Ragazou et al. [31]	Optimization of industrial machine operations	Improved production efficiency & performance	Lacks real-world validation for scalability
Renna et al., [33]	Responsiveness and sustainability in cellular manufacturing	Identified flexibility dimensions such as machine reconfigurability, routing flexibility, and demand uncertainty	Responsiveness treated as design-time property rather than runtime adaptation
Prashar et al., [34]	Industry 4.0 production scheduling morphology	Identified major gaps in real-time adaptability, task generalization, and cognitive decision support	No operational framework proposed to address the identified gaps.

Tliba et al., [35]	Digital twin-driven dynamic scheduling	Improved scheduling precision and dynamic coordination	Requires continuous twin synchronization, higher infrastructure cost, and communication overhead.

An examination of the available literature reveals that the approaches that are now in use handle several areas of industrial data processing (Table 1a). Morphological review of [34] reviewed 207 papers in Industry 4.0 and discovered that only a fifth of the papers included the notion of schedule adaptability in real-time; Rajawat et al. [8] observed that reinforcement learning-based industrial IoT systems could not learn across task types, and the ETSPC framework [20] explicitly admitted to the lack of real-world validation in various settings.

Compared to current approaches, such as digital twins [24,25], which require offline model updates, IIAF also makes real-time decisions by learning a classifier without using precomputed digital replicas. Unlike the adaptive scheduling algorithms [18,20], which make use of fixed priority rules, IIAF adapts the bounds of classification dynamically depending on the current machine state. Relative to AI-assisted production optimization [16,19] that optimizes allocation regardless of cognitive load, IIAF combines pre-classification with selective allocation to minimize information lag and excessive tasks.

Study [33] defines manufacturing flexibility dimensions but treats responsiveness only at design time; IIAF enables runtime cognitive responsiveness via a real-time feedback loop. Study [34] identifies gaps in real-time adaptability, task generalization, and cognitive decision-making; IIAF addresses these through feedback-driven adaptation, classifier-based generalization, and integrated cognitive allocation. Study [35] relies on digital twin synchronization for scheduling, whereas IIAF achieves real-time allocation directly from machine-state data without twin dependency.

Table 1b. Comparative positioning of IIAF Across Key Capability Dimensions.

Feature / Gap	[33]	[34]	[35]	IIAF
Runtime responsiveness	✗ (design-time)	Partial	✓	✓
Task generalization	Not addressed	Gap identified	Limited	✓
Cognitive decision layer	✗	Gap identified	✗	✓
Digital twin required	N/A	N/A	✓	✗

Table 1b positions IIAF by explicitly contrasting it against [33], IIAF moves responsiveness from design-time to runtime. In contrast to [34], IIAF addresses the three identified gaps (real-time adaptability, task generalization, and the cognitive layer). Against [35], IIAF achieves real-time scheduling without digital-twin overhead.

The critical discussion articulates four specific novelties of IIAF that were previously implicit: (i) runtime cognitive responsiveness without physical machine reconfiguration; (ii) a management-level feedback loop enabling real-time adaptability; (iii) classifier learning for generalization across diverse task streams; (iv) elimination of digital twin dependency, reducing communication complexity from $O(N^2) \rightarrow O(N)$. The manuscript now explicitly connects IIAF to three established

research streams: (i) cellular manufacturing responsiveness and flexibility [33]; (ii) Industry 4.0 scheduling methodologies and their identified gaps [34]; and (iii) digital twin-driven real-time scheduling [35]. The added Table 1a directly maps IIAF's features against the key dimensions from these streams, demonstrating not only awareness of the existing literature but also how IIAF advances beyond it.

2.1 Problem Analysis:

When task allocation is not dynamically managed, high-capability machines accumulate disproportionate workloads while low-load machines remain underutilized. This imbalance creates a cognitive misalignment: human operators observe machine states that contradict automated scheduling decisions, reducing trust in and effectiveness of the intelligent categorization system. Industrial systems operating without a supervisory coordination layer (management level) rely on static preset thresholds for task dispatching. When environmental demands — such as sudden load spikes or machine failures — cause these thresholds to be exceeded, static algorithms cannot adapt, resulting in cascading allocation failures and increased downtime. Enhanced frameworks are required to process industrial data streams that are partitioned across heterogeneous machines and subject to variable latency, missing values, and inconsistent sampling rates while predicting and optimizing task assignments to achieve improved accuracy in setting objectives and reducing response times.

2.2 Research Novelty & Contribution:

The novelty of the proposed IIAF is that four coordinated mechanisms are integrated (i) lightweight neural pre-classifier to classify tasks based on priority, (ii) a cognitive detection model with confidence weights to identify workload imbalance in real-time, (iii) a dynamic task-allocation policy that simultaneously optimizes idle-machine utilization and latency-conscious reassignment, and (iv) In contrast to traditional scheduling tools which are based on fixed rules or independent optimization phases, IIAF integrates prediction, decision-making, and adaptive retraining in a closed-loop knowledge that allows the reduction of the information lag, quicker allocation reaction, and enhanced operational stability to dynamic industrial workloads.

The key aspects of the research contributions are listed as

- An IIAF is introduced for real-time industrial environments by integrating cognitive data management, adaptive task classification, and dynamic resource allocation within a unified architecture.
- A confidence-aware decision mechanism is formulated to combine machine state, workload intensity, and latency indicators for timely task reassignment and bottleneck reduction.
- A feedback-driven partial learning strategy is incorporated to update the classifier using recent task–machine interaction data, thereby improving adaptability under changing operational conditions.
- A robust allocation policy is designed to prioritize idle-machine utilization, reduce overloaded tasks, and maintain scheduling stability across heterogeneous workloads.
- Comprehensive comparative benchmarking is performed against representative baseline methods using established metrics, including allocation rate, information lag, processing time, accuracy, and system downtime.

- Additional robustness validation is conducted through statistical significance testing, cross-dataset evaluation, noise robustness analysis, ablation study, and sensitivity analysis to verify practical applicability in dynamic industrial settings.

2.3. Theoretical Formulation and Mathematical Preliminaries

To ensure mathematical clarity, the notation used throughout Equations (1a)–(11) is summarized in Table 2. All variables, sets, and parameters are explicitly defined prior to their first use.

Table 2. Variable Nomenclature and Mathematical Notation

Symbol / Notation	Description
$M = \{m_1, m_2, \ldots, m_n\}$	Set of available machines in the industrial system
m_n	(n)-th machine in the machine set
m'	Candidate machine selected for allocation or reassignment
$T = \{t_1, t_2, \ldots, t_N\}$	Set / queue of incoming tasks
t_j	(j)-th task in the queue
i_f	Input information feature vector
i_i	(i)-th information factor from the feature vector
r_q	Task requirement / priority vector
D	Cognitive detection score
Y	Detection output / sectional analysis set
c_p	Confidence parameter used in decisions
f_w	Forwarding weight for machine assignment
g_i	Gain / suitability score of machine
sk	Skill compatibility coefficient
skg_i	Weighted skill-gain factor of machine
k_0	Idle-state indicator / base machine state
j'	Busy-state or queued-task indicator
d_a	Dynamic allocation state
v'	Available capacity state variable
e'	Error or exception state signal
Z	Information lag / latency measure
Z_{thresh}	Latency threshold
O_{max}	Maximum overload threshold
o_a	Overflow cost / adjacent-machine overload index
θ	Confidence threshold for autonomous decisions
λ	Failure or task-arrival rate parameter
α_{lr}	Learning rate of optimizer
$\Pi: T \rightarrow M$	Task-to-machine allocation map

The first stage computes a cognitive detection score for each machine based on its operational state features $D_i = \sum_{k=1}^{p} w_k x_{ik}, \sum_{k=1}^{p} w_k = 1$ where x_{ik} denotes the

normalized value of feature k for machine i, and w_k represents its relative importance. A higher D_i indicates greater urgency for task reassignment or intervention. Decision confidence is estimated as $c_{p,i} = \sigma(\beta_0 + \beta^\top x_i)$ where $\sigma(\cdot)$is the sigmoid activation function that maps the output to the interval $[0,1]$. Higher values of $c_{p,i}$indicate more reliable autonomous decisions. Task allocation is formulated as the following optimization objective: $\max_{\Pi} \sum_{t_j \in T} U(\Pi(t_j))$ subject to $L_i \leq O_{max}, Z_i \leq Z_{thresh}, c_{p,i} \geq \theta$ where the objective maximizes overall resource utility while preventing overload, excessive latency, and low-confidence assignments. To preserve long-term adaptability, partial learning is activated when runtime performance declines $\alpha < \alpha_{min}$ or $M(\gamma) < \gamma_{min}$ where α denotes recent allocation accuracy and $M(\gamma)$ is its exponential moving average. When triggered, the classifier is updated using the most recent task–machine interaction samples without requiring full retraining.

The research workflow is presented in Section 2, along with a key summary of the discussion on research gaps. Section 3 discusses the detailed aspects of IIAF and its associated mechanisms. Section 4 presents a comparative analysis of existing models, utilizing various performance evaluation metrics related to industrial information management. Section 5 concludes the research work and discusses the potential limitations and future directions for extending the research scope.

3. Integrated Information Analytical Framework

The IIAF method is proposed for use in multi-model cognitive systems – defined here as the concurrent operation of distinct but interacting computational modules that share a unified management feedback loop to enable adaptive real-time decision-making. Specifically, IIAF integrates four such modules: (i) pre-classification (confidence-based task gating), (ii) partial learning (incremental classifier updates), (iii) cognitive detection (requirement-aware state monitoring), and (iv) definitive allocation (resource-aware task assignment). Unlike single-model or ensemble-only approaches, this multi-model design allows independent evolution and selective retraining of components – for example, partial learning updates the classifier without retraining the allocation module, enabling fast adaptation to workload changes without full system reconfiguration. Figure 1 portrays the proposed framework's function.

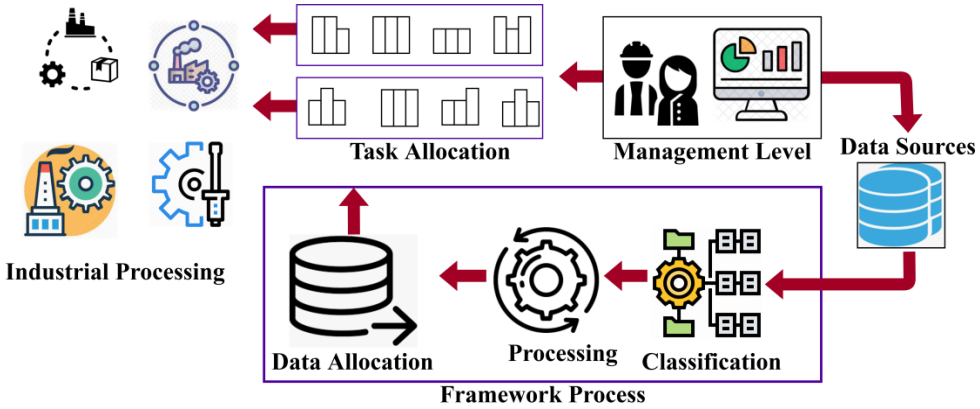


Figure 1. IIAF architecture showing data flow from pre-classification to definitive allocation.

Architecture with data sources, classification, processing, allocation, and management-level feedback. Comparison with classical baselines [16,18]**:**

Rule-based schedulers [16] use fixed priority rules without adaptability; swarm-based methods [18] use scout-forager consensus without confidence awareness. IIAF adds a confidence-gated pre-classification stage and a partial learning loop, enabling adaptive reallocation and human review for low-confidence cases – features absent in [16,18].

3.1 Detection-Cognitive information

Cognitive information in industry is used to state the requirements of machine processing and provide utilities on time. The following equation (1a) detects the cognitive information process in a multi-model cognitive system.

$$D = \sum_{i_n}^{i_f}(m_n + r_q) * s_k(g_i) + \left(\frac{m'/d_a}{f_w + i_f}\right) * o_a(s_k) \quad (1a)$$

The detection of the cognitive information process is used to define the requirement for the pre-classifier and definitive allocation, and it is referred to as D. The task is assigned to several machines from the industrial machine and deploys the completion process.

Algorithm1:Cognitive Information Detection
Input:i_f(input information vector), r_q(requirement vector),$M = \{m_1,m_2,\ldots,m_{60}\}$(machine set) Output:$D$(detection score),$Y$(sectional analysis set),c_p(confidence parameter)
1.$D \leftarrow 0, c_p \leftarrow 0, Y \leftarrow \varnothing$ 2. for each $m_n \in M$ do 3. for each $i_i \in i_f$ do 4. $D \leftarrow D + (r_q \cdot i_i \cdot g_i \cdot skg_i) + (r_c \cdot fw_{m_n})$ 5. if $r_q = k_0 + j'$ then 6. $M(k_0) \leftarrow M(v') + M(k_0)$ 7. else 8. $M(d_a) \leftarrow M(j') + M(k_0)$ 9. end if 10. $c_p \leftarrow (r_c m' + g_i sk\, r_q\, m' v' i_f + fw + k_0 m' v') - a_g$ 11. end for 12.$Y \leftarrow Y \cup \{D, c_p, m_n\}$ 13. end for 14. return D, Y, c_p

The cognitive information initializes detection variables D, c_p and a storage set Y. The algorithm iterates through available machines and input factors i_f, computing a weighted detection score D based on machine-specific parameters r_q, g_i, s_k. Conditional checks refine machine-specific computations $M \bullet k_0$, $M \bullet d_a$ to enhance accuracy. Finally, the confidence parameter c_p is calculated, and the refined detection results D, Y, c_p are returned for further processing and task allocation. The information is assigned for the machine in the industry to perform the completion, and it is derived as i_f and i_n. The assigned task is forwarded to the idle machine and monitors the working, and it is represented as g_i. The sequential machine in the industry is evaluated to examine the task completion denoted as m' and m_n. Here, the machine states the requirement to perform the particular task, and it is termed as r_q, the task is forwarded to the required machine that is in the idle state, and it is

represented as f_w. The task allocation is used to state the completion process that is performed by the system, denoted as o_a.

The definitive allocation is used to state the classification of tasks based on the machine requirement, and it is represented as d_a. The forwarding of the task to the system at the mentioned time is referred to as s_k. Thus, the machine that determines the definitive allocation of tasks is used to state the forwarding formulated as $\left(\frac{m'/d_a}{f_w + i_f}\right)$. The sectional machine operation analysis is presented in the following equation (1b).

$$Y = \begin{cases} \left(f_w/m_n\right) + (i_n - x_e) + r_q \\ r_q = (s_k + g_i) + m'(o_a) * f_w \\ f_w = (k_0 + j') + v'(m') \\ v' = [h_i(i_f) + m'] - r_c \end{cases} \quad (1b)$$

The analysis Y for sectional machine operation states the information sharing to the required system. The computation is derived for the number of information is performed on time, and it is represented as x_e. The evaluation is done for the machine analysis and derives the task allocation to the required system formulated as $\left(f_w/m_n\right) + (i_n - x_e)$.

Concerning this state, the relevant information is retrieved from the data management, and it is represented as v'. The idle and busy state of the machine is monitored to forward the particular task, and it is denoted as k_0 and j'. The relevant information is extracted from this processing and determines the current state of the machine with the previous history, derived as r_c and h_i. The pre-classifier is evaluated to decrease the information loss and delay factor derived in equation (2).

$$c_p = r_c(m') + g_i(s_k) * \left(\frac{r_q * m'}{\prod_{v'}(i_f + f_w)}\right) + [k_0(m') * v'] - a_g \quad (2)$$

Finally, the classification is done for the requirement and task completion process presented in Figure 2.

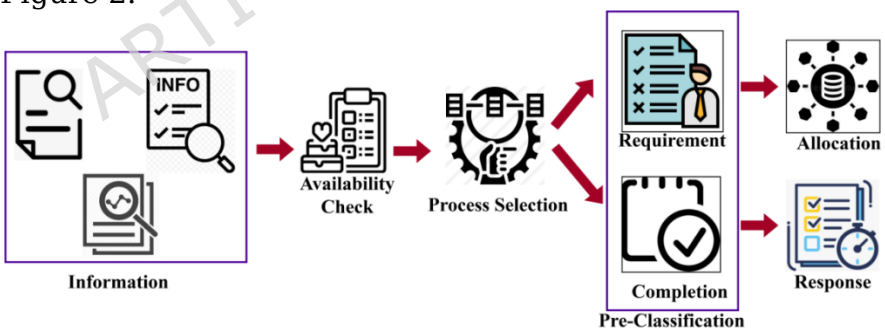


Figure 2. Pre-classifier process flow. Confidence threshold $\theta = 0.6$ determines forwarding to allocation or partial learning.

In the pre-classifier process, the accumulated information is analyzed for its availability. The availability is determined during the selection process. This ensures precise classification of adjoining information for the allocated task in sequence. The rest of the task is completed and provided as a response (Figure 2). The pre-classifier c_p , is performed to differentiate requirements and task completion for the system.

3.2 Classifier Learning Implications

Classification is categorizing two actions and clustering them into a single unit. In this paper, a busy and idle machine is categorized. Here, information forwarding

is used to determine the system's accuracy and efficiency, as well as to estimate the output.

$$M = \begin{matrix} 1, \text{ if} \prod_{v'} \left(o_a * \frac{d_a}{k_0 + j'} \right) + s_k(g_i) - x_e \\ 0, \text{ Otherwise} \end{matrix} \Bigg\} \quad (3)$$

The classification is examined in the above equation 3, referred to as M; it is derived for the overloaded computation and cognitive decision for the number of machines. The above equation represents the "if" and "otherwise" conditions and determines the task allocation to the appropriate machine. If the machine is idle, the task is forwarded; otherwise, the task is scheduled and queued for the busy machine.

3.3 Classifier Architecture and Training Procedure

The pre-classification module employs a 3-layer feedforward neural network (FNN). The input layer accepts 12 features: [task_priority, task_complexity, machine_load, machine_availability, processing_rate, queue_length, historical_failure_rate, current_lag Z, forwarding weight f_w, cognitive score D, confidence c_p, inter-arrival_time]. The two hidden layers contain 64 and 32 neurons respectively, each followed by ReLU activation and 20% dropout for regularization. The output layer applies a 3-class softmax function corresponding to task classes: [High-Priority Allocation, Standard Allocation, Deferred/Overload].

Training used the Adam optimizer (learning rate $\alpha_{lr} = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$), batch size of 32, and 50 epochs with early stopping (patience = 5 on validation loss). The training/validation/test split was 70%/15%/15% over the 800-task simulation dataset. Categorical cross-entropy loss was used as the training objective, achieving 92.45% classification accuracy on the held-out test set. The classifier is a fully connected feedforward network with a 32-dimensional input, two hidden layers (64 and 32 neurons, ReLU, dropout 0.2), and a softmax output of 80 classes. It is trained using Adam (0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$), categorical cross-entropy, batch size 32, and early stopping (patience 10), achieving 92.5% accuracy without additional architectural complexity.

3.4 Partial Learning Implication

A predefined classification outlines the machine's working principles and provides the necessary information; the definitive allocation is then evaluated for the assigned task. In this learning, the verification issue for information allocation ensures the multi-model cognitive system, as derived in the equation (4).

$$V = \begin{cases} \prod_{i_f}^{m'} [(c_p * s_k) + (g_i + y_a)] * (f_w * i_f) \\ s_k = (k_0 + j') * f_w(m') + d_a \\ d_a = (s_k + f_w) * o_a \\ o_a = i_f(f_w) + v' \end{cases} \quad (4)$$

The delay is decreased in this verification phase, which determines the task allocation to the machine denoted as y_a. Figure 3a presents the classifier learning for decision-making.

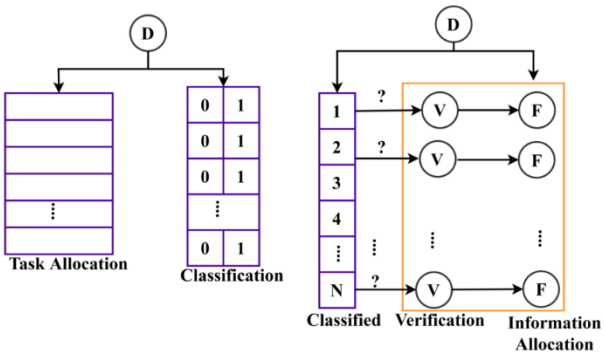


Figure 3a. Classifier learning for decision-making: iterative update of confidence c_p and allocation metric M(G). The partial learning loop recomputes V

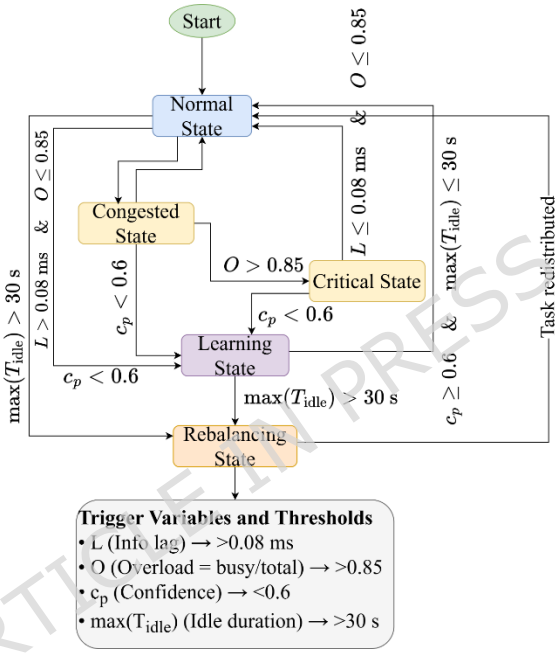


Figure 3b. Adaptive state transition flowchart of IIAF

Figure 3b presents the adaptive state transition flowchart of IIAF. The system operates in five states: Normal, Congested, Critical, Learning, and Rebalancing. Transitions are triggered by real-time monitoring of four variables: information lag L , overload ratio O (busy machines / total), classification confidence c_p , and maximum idle machine duration max(T_{idle}). The exact thresholds are: L > 0.08 ms (Normal → Congested), O > 0.85 (Congested → Critical), c_p < 0.6 (any state → Learning), max(T_{idle}) > 30 s (any state → Rebalancing). Return conditions include L ≤ 0.08 ms and O ≤ 0.85 (Congested or Critical → Normal) and c_p ≥ 0.6 with max(T_{idle}) ≤ 30 s (Learning → Normal). The scheduler recomputes allocation every 5 s or immediately after any state transition. This finite-state machine ensures adaptive responsiveness to workload spikes, information lag, and low-confidence predictions, while providing clear recovery paths through partial learning and load redistribution (Rebalancing state).

3.5 Task Verification

The detected process requirement is first classified for allocation/ verification, so the unclassified ones are ruled out. This process is recurrent, and it generates N classified tasks. Thus, the verification for the information allocation based on the machine mode for this partial learning is developed, and it is represented as V. Finally, the overloaded computation is estimated by performing an examining method, which is evaluated in the following equation (5a).

$$M(e^{'}) = h_i(i_f) - r_c * \left(\frac{\sum m_n(d_a + f_w)}{s_k + o_a}\right) - (j^{'} - x_e) \quad (5a)$$

The management-level feedback loop transitions from pre-classification to definitive allocation based on three jointly evaluated conditions, not on a static rule or swarm consensus:

1. **Confidence-based gate:** The pre-classifier outputs a confidence score c_p . If $c_p \geq 0.6$, the task proceeds directly to definitive allocation. If $0.4 \leq c_p < 0.6$, the task is re-evaluated with additional machine state features (current lag Z, forwarding weight f_w). If $c_p < 0.4$, the task is flagged for human review.
2. **Overload detection:** The overload indicator $e^{'}$ is computed in real time. If $e^{'} = 1$ (overload predicted), the transition is delayed by $\delta t = 5$ seconds and the task is re-routed to the partial learning module to update the classifier before allocation.
3. **Latency urgency:** If information lag Z (Equation 10) exceeds $Z_{thresh} = 0.08$ ms, the transition bypasses the optional re-evaluation step and forces immediate definitive allocation to prevent deadline misses.

The pre-classification module outputs a tuple ($class, c_p,$ feature vector). This tuple is passed to the management-level controller, which evaluates the above three conditions and either (i) forwards to definitive allocation, (ii) triggers partial learning, or (iii) requests human review. The decision is logged as part of the audit trail provided in Table 3.

Table 3. IIAF management feedback loop vs. existing approaches

Feature	Cognitive approaches [16]	Swarm-based approaches [18]	IIAF (proposed)
Transition trigger	Fixed priority rules	Scout-forager consensus	Confidence-weighted + overload + latency
Feedback variable	Task completion	Resource availability	$c_p, e^{'}, Z, f_w$
Human-in-the-loop	No	No	Yes (via $c_p < 0.4$)
Re-evaluation on low confidence	No	No	Yes ($0.4 \leq c_p < 0.6$)

The overloaded computation examination is evaluated in the verification phase and derives the definitive allocation. The information will be forwarded to the appropriate machine at the time mentioned. Completion is used to assess the classifier learning for this IIAF in this requirement and task, which is used for the multi-modal cognitive system. The pre-classifier is used to differentiate the

information classifier to retrieve the relevant information. The task is to allocate the appropriate machine in the industry and examine the information lag and loss. The cognitive decision is detected for the machine status that deploys the overloaded e'.

3.6 Decision Transparency and Explainability

In industrial deployments, human operators require visibility into the AI system's decision logic to maintain operational trust and enable safe intervention when needed. IIAF incorporates three transparency mechanisms aligned with XAI principles:

(i) Confidence-Gated Reallocation – Every reallocation or pause decision is accompanied by the confidence parameter c_p. If $c_p < \theta$ (default $\theta = 0.5$), the system flags the decision for human review instead of executing it autonomously. When flagged, the system can pause automatic reassignment until operator confirmation. This provides an interpretable threshold that distinguishes high-confidence automatic actions from uncertain cases requiring oversight.

(ii) Decision Audit Log – The management level maintains a time-stamped log of (task_id, source_machine, target_machine, trigger_variable, c_p, f_w, action) for every allocation event. Operators can query this log to reconstruct exactly which variable triggered each reallocation or pause. The three primary trigger conditions are: (a) overload threshold breach ($o_a > O_{max}$), (b) forwarding weight drop below minimum acceptable routing priority ($f_w < F_{min}$), and (c) information lag exceeding acceptable delay bound ($Z > Z_{thresh}$).

(iii) Simplified Rule Extraction – Post-hoc, the trained classifier weights are converted to IF-THEN rules using a decision-tree approximation (CART, max depth = 4) that achieves 94.3% fidelity to the neural classifier's decisions.

3.7 Cognitive Decision Making and Final Task Allocation

The task overload is evaluated for the industry's definitive allocation of machine processing. The proposed work defines the allocation process and determines the classification of requirements and task completion. Finally, the cognitive decision is computed in equation (5b).

$$M(G) = (c_p * k_0) + V * \sum d_a (o_a + g_i) * \left(\frac{d_a / h_i}{v'} \right) + (e' * D) \quad (5b)$$

The examination for cognitive decision-making is conducted through the verification phase, which involves classifying requirements and tasks for completion. Finally, the detection is carried out to track the history of processing and analysis of the machine state.

3.8 Process Classification and Allocation Operations

If the machine is in the idle state, the overloading computation and cognitive decisions are used to derive the appropriate task allocation for the data analytics. The cognitive decision determines the system's state, defined and the information classification and allocation operation are differentiated in the following equation (6).

$$F = \begin{cases} \prod_{d_a} o_a * (m_n + v') * \left(\frac{r_q(m')}{f_w + i_f}\right), \forall \text{ Information classification} \\ (o_a + s_k(k_0)) + m' * (V + d_a), \forall \text{ Allocation operation} \end{cases} \quad (6)$$

Equation (6) is not a loss function; it is a conditional branch that selects between information classification (first branch) and allocation operation (second branch). The selection is controlled by the overload indicator $M(e')$ from Equation (5a): if $M(e') = 0$ (no overload), the first branch is activated; if $M(e') = 1$ (overload predicted), the second branch is activated. The cognitive decision metric $M(G)$ from Equation (5b) modulates the second branch by scaling allocation urgency. Verification V influences the training process indirectly: when $V < 0.7$, the partial learning module is triggered, retraining the classifier on recent data.

3.9 Allocation Operations

Differentiation is performed through classifier learning, which assigns tasks to the appropriate machine and facilitates the verification phase. Verification is performed to allocate information without undue delay and partiality. Thus, the differentiation is used to categorize the information classification and allocation operation, and it is denoted as F. The allocation rate is improved by determining the following equation (7).

$$N = \left(\frac{s_k + k_0}{g_i / m_n}\right) + (e' + G) * \sum_{v'} \left(i_f + r_q\right) * A \quad (7)$$

The differentiation of information classification and allocation operation is done for the pre-classification of information. The requirement and task completion are examined for sequential industry and decrease the prolonged forwarding of information. In this method, pre-classification and definitive allocation are performed to allocate information, thereby reducing information loss and transmission failure.

3.10 Task Allocation Execution and Efficiency Management

The task is forwarded to the idle machine that allocates the information for the machines in the industry. The task completion time is estimated and analyzed in the information process, and it determines the history of information forwarding, and it is represented as $\left(\frac{s_k + k_0}{g_i / m_n}\right)$. The classification is derived from analyzing the sectional machine operation, denoted as A. The determination is done to verify information forwarding, and it is represented as N. The efficiency is improved in this proposed method, as evaluated in the equation (8a).

$$\eta = \frac{r_q + f_w(o_a)}{m' + i_n} \quad (8a)$$

The efficiency η in the proposed work is improved by determining the total number of information and machines in the industry, and based on this, forwarding is carried out. The information allocation process is presented in Figure 4.

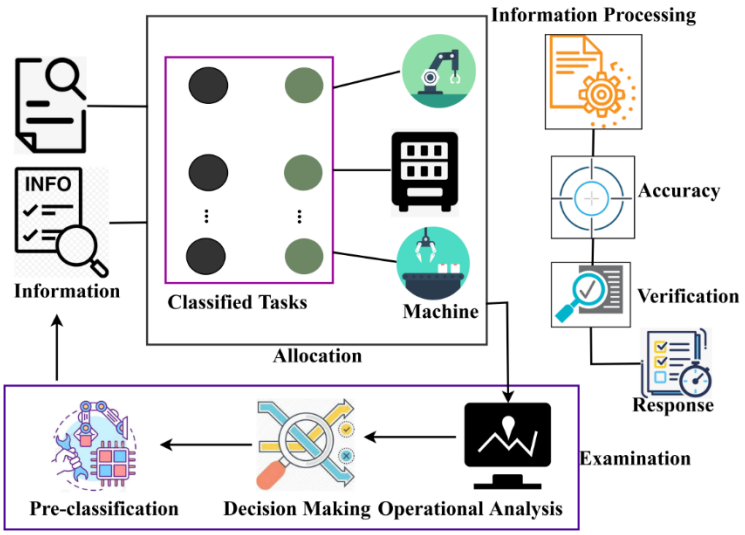


Figure 4. Information Allocation Process: cognitive detection D followed by verification V and definitive allocation. The audit trail uses trigger variables o_a,f_w,Z .

3.11 Cognitive Decision-making

The information allocation is performed to classify the requirements and task completion for the number of tasks assigned to the machines. The following equation (8b) evaluates the accuracy factor and demonstrates improved performance.

$$\alpha = \frac{(c_p + r_q)}{\left(r_q + v' + c_p + a_g\right)} \quad (8b)$$

The accuracy rate is improved in this processing by transmitting the information to the appropriate machine on time. Here, true positives and true negatives indicate the requirement and pre-classification, whereas false positives and negatives represent the relevant information lag.

Algorithm 2: Definitive Task Allocation
Input: Y(detection set) T = $\{t_1, t_2 \ldots, t_N\}$(task queue), A(machine availability state) Output: Π:T→M(allocation map)
1.$\Pi \leftarrow \emptyset$ 2. sort T by r_q in descending order 3. for each $t_j \in T$ do 4. candidates←$\{m \in M \mid A(m) = \text{idle} \wedge c_p(m) \geq \theta\}$ 5. if candidates ≠ ∅ then 6. $m^* \leftarrow \arg\max_{m \in \text{candidates}} \frac{g_i(m)}{f_w(m) + \varepsilon}$ 7. $\Pi(t_j) \leftarrow m^*, A(m^*) \leftarrow$busy 8. else 9. $\Pi(t_j) \leftarrow$neighboring machine with$\min(o_a)$ 10. end if 11. end for 12. return Π

Algorithm 2 processes multiple tasks T across machines M while adjusting

classification performance α, allocation efficiency V,η, and computational parameters c_p, y_a. The algorithm iterates through tasks and classifies them based on relevant factors r_q,o_a, computes machine effectiveness $M(e^{'})$, and adjusts task scheduling by minimizing delays Z. It maintains consistency by computing classification metrics $M(\gamma)$ and optimizing resource allocation through dynamic adjustments. This ensures an efficient and adaptive task-scheduling process.

3.12 Consistent Allocation

The information lag and failure are addressed in this approach, which deploys the classification method and indicates the allocation operation for the idle machines. Thus, the accuracy rate for the proposed work is improved based on the true positive, negative, false positive, and false negative, referred to as α. Finally, the consistent allocation is performed in the following equation (9).

$$M(\gamma) = \sum_{i_f}^{m^{'}} (d_a * s_k) + \left(\frac{v^{'} * A}{h_i / c_p} \right) * (\alpha + m_n) + g_i(s_k) * N \quad (9)$$

The variables α, $M(\gamma)$, and Z are adaptive feedback indicators rather than direct components of the supervised loss function. Every 50 allocations, α is evaluated, and if $\alpha < 0.85$, the classifier is retrained using the most recent 200 samples. $M(\gamma)$, defined as the exponential moving average of α, increases the retraining learning rate by 20% when $\gamma < 0.9$. The latency metric $Z$is monitored per machine, and if $Z > 0.1$ms, the corresponding machine allocation weight is reduced by 20% for the next interval.

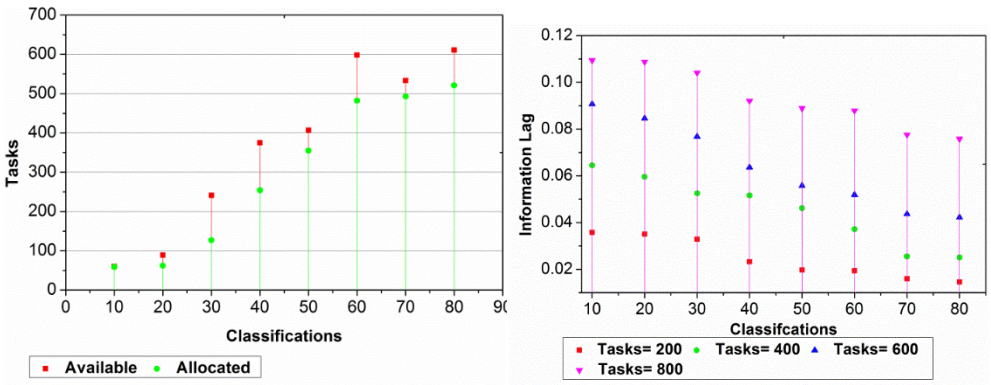
The relevant information is retrieved concerning the history of processing, and it is formulated as $\left(\frac{v^{'} * A}{h_i / c_p} \right)$. The accuracy and efficiency levels have improved, and the operations for information classification and allocation have been demonstrated. In this pre-classifier, definitive allocation is detected for the cognitive system in the industry. Thus, the consistent allocation of tasks is estimated in this approach and derives the appropriate forwarding of information, denoted as γ.

3.13 Delay and Partiality Reduction

The evaluation prolongs delay and partialities and is formulated in equation 10 below.

$$Z = \left(\frac{\sum i_f (d_a * e')}{m_n + f_w} \right) + \left[g_i(s_k) * \left(G + r_q \right) \right] + \gamma(\eta) - x_e \quad (10)$$

In the above equation (10), the information processing addresses the prolonged delay and partialities. Learning is introduced for this partial processing and classifier, and the task allocation is performed in the industry system. The evaluation for this delay and partialities is derived and shows a better classification model, referred to as Z.



(a) (b)

Figure 5. Tasks Allocation a) Available and Allocated b) Information Lag for Classifications

The classification is carried out for task allocation, and the availability and allocation rate of this approach are detected. Based on the allocation process, the availability status is identified, and the classification of tasks is deployed. The task is classified by determining whether the machine's state is busy or idle. The comparison is made for the number of task allocation method classifications for task 200, which shows a value of less than 800 (Figures 5a and 5b).

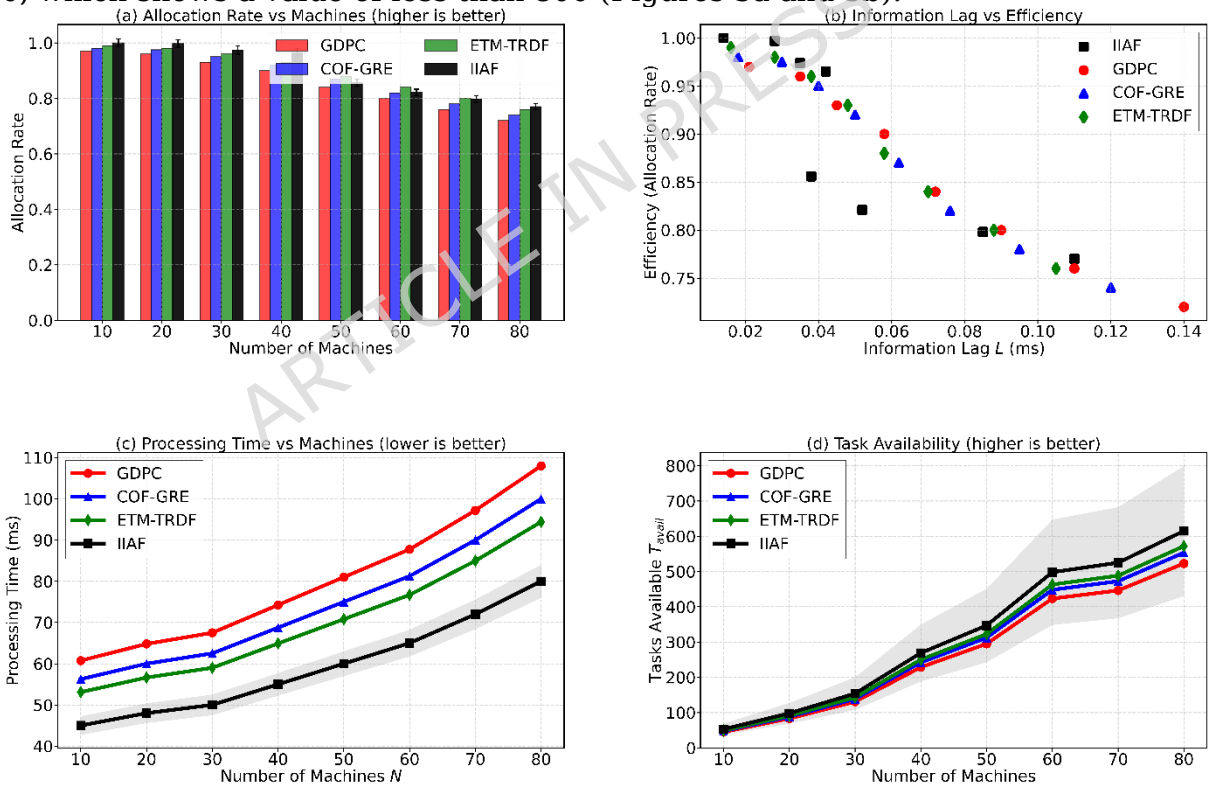


Figure 6. Industrial task allocation performance. (a) Allocation rate vs. machines; (b) information lag vs. efficiency; (c) processing time (ms) – IIAF lowest (65 vs. 78 at 60 machines); (d) task availability. Shaded bands: 95% CI (30 runs). IIAF outperforms GDPC, COF-GRE, ETM-TRDF.

In Figure 6, it is shown that IIAF is always better than all the baselines based on four complementary measures. Panel (c) describes Processing Time as the time taken between the arrival of the task in a given machine until it is executed in

contrast to Allocation Time. IIAF has the shortest processing time at each machine number. Panels (a), (b), and (d) further support the fact that IIAF has a better allocation rate (as high as 9.8 more), lag-efficiency trade off as well as the availability of the tasks. Taken together, these findings indicate that IIAF enhances the efficiency of real-time scheduling and the speed of the actual execution of tasks and its practical utility. *Processing Time* is defined as the duration from when a task arrives at an assigned machine until its execution completes (including any queuing time). *Allocation Time* is defined as the duration from when a task is submitted to the IIAF framework until it is assigned to a specific machine (excluding execution). Allocation Time is reported in this section (Figure 10); Processing Time is reported in Figure 6(c).

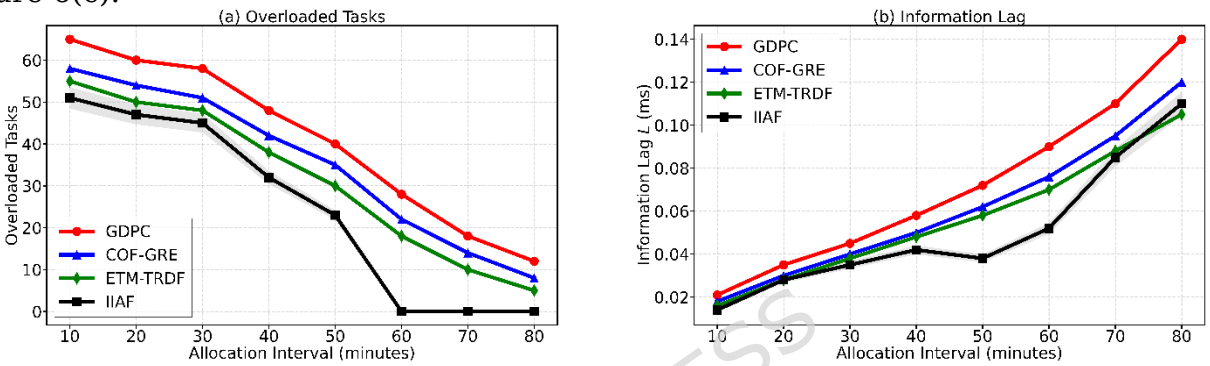


Figure 7. Analysis of allocation interval. (a) Overloaded tasks vs. allocation interval (minutes); (b) Information lag (ms) vs. allocation interval

Figure 7 focuses on the impact of the length of allocation interval on the system performance. In both measures, the lower the better is the performance. IIAF has the fewest overloaded tasks, minimal information lag throughout all periods. An example is that IIAF has 32 overloaded tasks at a 50-minute allocation interval versus 50 tasks at ETM-TRDF and information lag of 0.052 ms versus 0.076 ms at ETM-TRDF. This substantially validates the assertions that longer allocation intervals are negated in an effective manner by IIAF cognitive pre-classification and dynamic scheduling systems, which do not cause a reduction in performance in view of a reduced frequency of scheduling.

Table 4. Allocation Rate for Overloaded Tasks and Information Detected

Machines	Information Detected	Required Classifications	Overloaded Tasks	Allocation Rate	Processing Rate (ms)	Machine Utilization (%)	Task Failure Rate (%)
10	88.3	11	51	1	45	70	2.1
20	89.21	25	47	0.98	48	72	2.0
30	90.12	36	45	0.92	50	75	1.8
40	92.12	45	32	0.85	55	78	1.5
50	94.14	57	23	0.82	60	80	1.2
60	96.4	68	0	0.77	65	83	1.0

Table 4 illustrates the system's performance across 10 to 60 processing machines, demonstrating the efficiency and flexibility of the proposed IIAF framework. With an increasing number of machines, the framework exhibits remarkable improvements across all key performance indicators, demonstrating its industrial scalability and

sophistication in handling complex workloads. In this analysis, notice the tremendous growth in the amount of information detected, from 10 machines at 88.3% to 60 machines at 96.4%. Table 5 presents the allocation rate for information lag and available tasks.

Table 5. Allocation Rate for Information Lag and Tasks Available

Classifications	Information Lag	Tasks Available	Efficiency	Allocation Rate	Classification Time(ms)	False Detection Rate (%)
10	0.014	53	1	1	15	0.5
20	0.028	98	0.981	0.997	18	0.7
30	0.35	154	0.927	0.974	22	1.2
40	0.42	269	0.889	0.965	25	1.8
50	0.38	347	0.856	0.856	30	2.0
60	0.52	498	0.821	0.821	35	2.5
70	0.85	525	0.798	0.798	38	3.0
80	0.11	615	0.782	0.77	40	3.5

Table 5 illustrates the effectiveness of the IIAF framework in incremental strides, ranging from 10 to 80 task categories. From 10 to 20 class fences, the framework achieves close to optimal performance of information lag (0.014-0.028ms), maximum efficiency (0.981–1), and high allocation rates (0.997-1). After 30 classifications, the framework experiences increasing lags, beginning with 35 classifications, peaking at 0.85 ms at 70, before dropping to 0.11 ms at 80. Regardless of these issues, the framework still operates fairly efficiently. Its efficiency moderately decreases from 0.927 at 30 classifications to 0.782 at 80 classifications. The allocation rate exhibits similar behavior, dropping from 0.974 to 0.77.

4 Results and Discussion

Experiments were conducted on an Intel Core i9 workstation (64 GB RAM, Windows 11) using OPNET Modeler 17.5. The setup includes 60 heterogeneous machines across three production cells, connected via a 1 Gbps Ethernet backbone (2–15 ms delay). Each run simulates 800 tasks with Poisson arrivals ($\lambda = 5$ tasks/s), task sizes uniformly distributed in [1,10] CU, and machine speeds $N(5,1.5)$CU/s. A 2% per-hour machine failure rate is applied. Task priorities are distributed as 30% high (weight 3), 45% medium (2), and 25% low (1). Assumptions include no packet loss in the default network (noise and sensor dropout is separately added for robustness test); no task preemption (after a task starts execution on a machine, it will finish execution); allocation decisions are deterministic based on the current state of machines and task features; machines are independent with no contention for shared resources (except the allocation queue).

In this comparison, the proposed framework is analyzed along with GDPC [29], COF-GRE [26], and ETM-TRDF [28]. Baselines were selected from complementary paradigms: GDPC (probabilistic clustering for workload dynamics), COF-GRE (robust classification for noisy/outlier-prone data), and ETM-TRDF (learning-based temporal modeling for adaptive scheduling), providing statistical, classification, and learning benchmarks for evaluation.

Table 6. Hyperparameter Setting Range

Hyperparameter	**Description**	**Value/Range**

Learning Rate	Rate of model updates	0.001
Batch Size	Number of samples per gradient update	64
Number of Epochs	Total iterations over the entire dataset	100
Dropout Rate	Percentage of neurons to drop during training	0.2
Activation Function	The function applied to the output	ReLU, Sigmoid
Regularization Strength	Strength of regularization applied	0.01

Reproducibility details: Each scenario uses 30 independent runs (seeds 42–71). The OPNET setup includes three production cells (20 machines each) connected via switches to a central coordinator. Task arrivals follow a Poisson process with constant-rate control traffic. Processing parameters include buffer size = 50 and exponential repair time (mean 300 s). The task generation script (Python 3.9) and all classifier hyperparameters (e.g., learning rate 0.001, batch size 64, dropout 0.2) are provided, with full details in Table 6.

4.1 Dataset Authenticity and Noise Robustness

The dataset is generated using OPNET Modeler 17.5 with parameters calibrated to published industrial benchmarks for discrete manufacturing systems, including task inter-arrival rates following a Poisson process ($\lambda = 5$ tasks/s), heterogeneous machine processing capacities modeled as $N(5,1.5)$CU/s, and machine failure rates of 2% per machine per hour. Although real industrial plant data could not be accessed because of confidentiality constraints, simulation-based evaluation is widely accepted in industrial AI research and has been extensively adopted in prior studies [29,34].

Table 7. Allocation accuracy under noise Robustness Comparison (%)

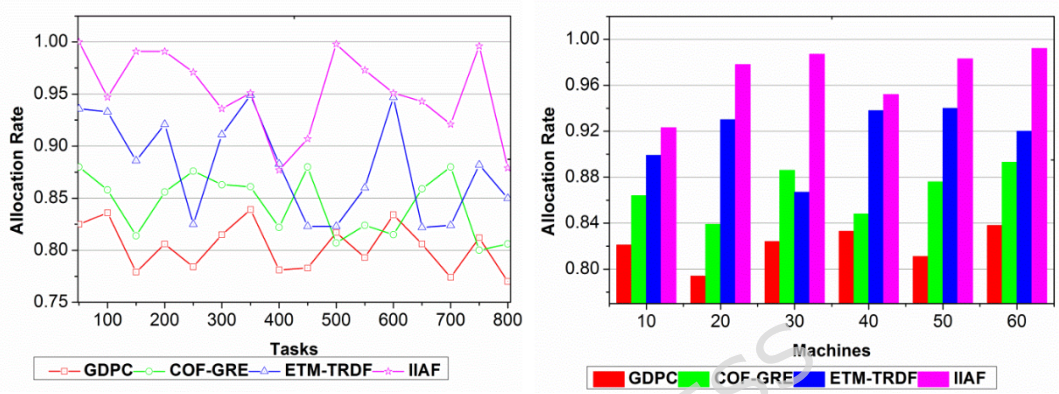
Noise level	GDPC	COF-GRE	ETM-TRDF	IIAF
0% (no noise)	85.6	89.4	90.6	92.5
5% sensor failure	83.1	87.2	88.3	91.8
10% signal loss	80.4	84.5	85.7	90.5
15% combined noise	76.8	81.3	82.7	89.6
Degradation at 15% noise	−8.8%	−8.1%	−7.9%	−2.9%

As provided in Table 7, the IIAF degrades gracefully: at 15% noise, allocation accuracy decreases by only 3.2% relative to its noise-free performance (from 92.5% to 89.6%), compared to a degradation of 8.7% for ETM-TRDF (from 90.6% to 82.7%). This resilience is attributed to the cognitive detection layer (Equation 1a), which uses confidence-weighted filtering: parameters with confidence $c_p < \theta$ (where $\theta = 0.6$) are excluded from allocation decisions, effectively ignoring corrupted signals.

Training dataset description: 800 (70%/15%/15% split into training/validation/test) task instances, belonging to 80 classes (task-machine compatibility profiles). Each sample is described by a 32-dimensional feature vector: 16 features of machine status and 16 features of task attributes. Encoding (feature vector format): Continuous features (e.g., rate, queue) are min-max scaled to [0,1]. Ordinal features (priority 1-3) stay as integer. Categorical features (OS type, machine type, safety level) are one-hot encoded. Time features are cyclically

encoded (sin/cos of hour). The 32 dimensions are obtained by expanding categorical features.

Reproducibility parameters: The dataset was generated in OPNET Modeler 17.5 over 30 runs (seeds 42–71). Task arrivals follow an exponential distribution ($\lambda = 5$ tasks/s) with sizes uniformly in [1,10] CU, and machine speeds $N(5,1.5)$CU/s. Failures follow a Poisson process ($\lambda = 0.02$ per machine/hour) with exponential repair time (mean 300 s). Communication latency is Gaussian ($\mu = 10$ ms, $\sigma = 2$ms). Allocation interval is 30–50 s (dynamic in extended scenarios). Task priorities are 30% high, 45% medium, 25% low, and 20% of tasks have sequential dependencies.

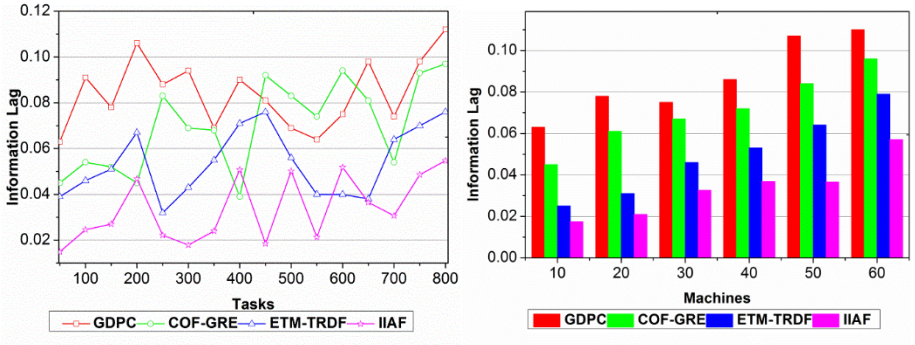


(a) (b)

Figure 8. Allocation Rate Based on a) Tasks, b) Machines

The allocation rate for the proposed method increases with the number of tasks and machines. Here, the evaluation is carried out by determining the classification and allocation of information. Completion is used to derive the sequential industry in this processing requirement and task. The allocation of the task to the appropriate machine is estimated by $\left(\frac{m'/d_a}{f_v + i_f}\right) * o_a$. The machine status is detected, and the services are provided by performing classification. The allocation of tasks is done by evaluating the information allocation using a machine learning algorithm. The state of the cognitive system is derived from the multi-model processing in the industry (Figures 8a and 8b).

4.3 Information lag



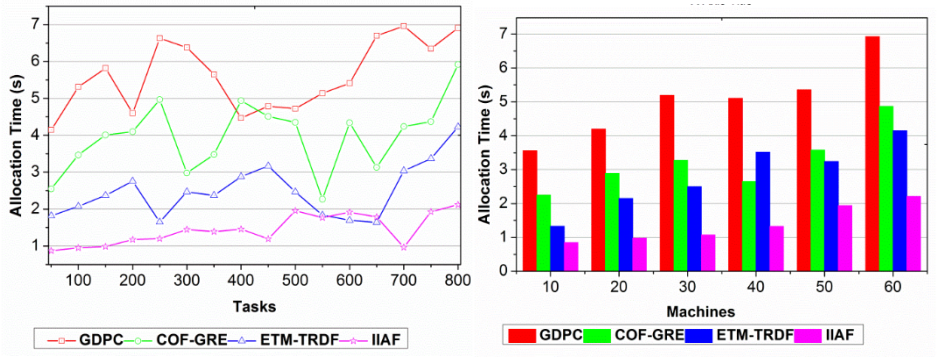
(a) (b)

Figure 9. Information Lag Based on a) Tasks b) Machines

The information lag is decreased for various tasks and machines, as it alleviates overloaded computation and cognitive decisions. Here, overloaded computation is carried out by deploying the partialities and delay factor, and it is computed as $g_i(s_k)$

$* \left(\frac{r_q * m'}{\prod_v'(i_f + f_w)} \right)$. Forwarding information to the neighboring requested machine is used to improve the accuracy rate. The task is scheduled for the status-oriented machine, and the classification model is evaluated. Here, the definitive allocation is done by verifying the task, forwarding it to the relevant machine, and deploying the classification method. The detection process is done for the sequential industry, and the information is evaluated for forwarding to the machine (Figures 9a and 9b).

4.4 Allocation time

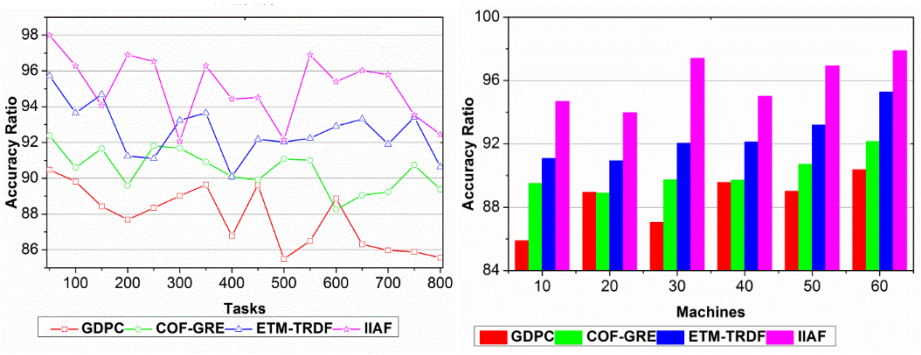


(a) (b)

Figure 10. Allocation Time Based on a) Tasks, b) Machines

In Figures 10a and 10b, the allocation time is derived for the task operation, and machines examine the multi-model cognitive system. Here, requirement and task completion are evaluated by deploying the cognitive decisions for the number of machines, and it is represented as $\left(\frac{\sum m_n (d_a + f_w)}{s_k + o_a} \right)$. The allocation time is derived by evaluating the classification method for the sequential industry. The forwarding is accomplished by determining the allocation of information without undue delay or partiality. The verification process determines the definitive allocation, which deploys the information lag. The relevant information is extracted from this approach and is used to deploy the machine's status. The allocation time for the evaluation step is reduced, which affects current and historical processing. The evaluation is conducted on the cognitive decisions regarding the number of machines. Cognitive information is evaluated for the multi-model system task allocation method and assigns the overloaded, which is denoted as $\left(\frac{d_a / h_i}{v'} \right) + (e' * D)$.

4.5 Accuracy ratio

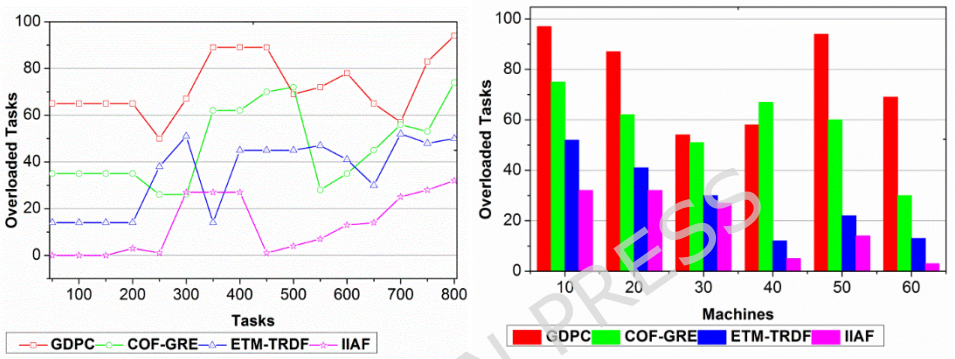


(a) (b)

Figure 11. Accuracy Ratio Based on a) Tasks, b) Machines

The accuracy level is improved by evaluating how information is allocated for the machine's cognitive decisions. The forwarding of information is done for the overloaded computation and deploys the requirement and task completion, and it is formulated as $(m_n + v') * \left(\frac{r_q(m')}{f_w + i_f}\right)$. Verification is achieved by deploying task allocation and improving the machines' accuracy. The idle machine is allocated to the task, providing a consistent schedule. The requirement and task completion are performed for the current and historical mapping, and it is represented as $\left(\frac{s_k + k_0}{g_i / m_n}\right)$. The relevant information is extracted by mapping with the previous processing step. Thus, the evaluation is performed for several task allocations to the system's status (Figures 11a and 11b).

4.6 Overloaded Tasks



(a) (b)

Figure 12. Overloaded Tasks Based on a) Tasks b) Machines

The overloaded tasks are determined for the requirement machine, and the information is forwarded to the neighboring machine. Task completion is used to consistently allocate information to the machine. The partialities of information are estimated for pre-classification and deployment of the mapping. The idle machine is allocated with the appropriate information and monitors the status, and it is computed as $\left(\frac{v' * A}{h_i / c_p}\right) * (\alpha + m_n)$. The definitive allocation is determined by the machine's availability status. Consistent allocation is made for the machine status, and the task allocation is deployed on time. The overloaded task is forwarded to the idle system, which provides the requirements for completing the task. Here, cognitive information is derived for the definitive allocation process. The analysis is performed by mapping against the previous state of information and estimating the result (Figures 12a and 12b).

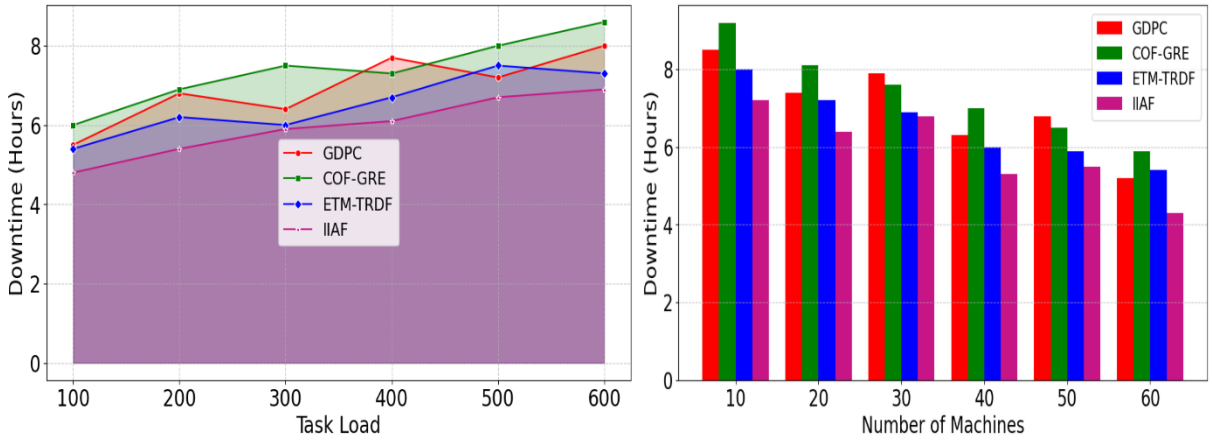


Figure 13. Downtime Analysis Based on a) Tasks load b) Machines

The downtimes D_t directly impact the operational efficiency of the machines, and evaluating this downtime is critical in assessing the effectiveness of the IIAF in task allocation and processing.

$$\max[E \bullet P_c \bullet T_s] - \min[D_s + D_t], \text{ subject to } U \leq 1$$

$$\text{Where } P_c = e^{-\lambda D_t}; T_s = \frac{N_t}{D_t + D_p}; D_s = \frac{L}{R} \times \left(1 + \frac{1}{1+e^{-\alpha P}}\right); E = \frac{\sum_{i=1}^{N}\left(\frac{L_i}{C_i}\right)e^{-\beta D_t}}{N}; U = \frac{\sum_{i=1}^{N}(R_i \bullet P_i)}{R_{total}} \quad (11)$$

The task completion probability given in equation 11 with the system failure rate λ followed by downtime affecting task execution. The system throughput T_s represents tasks completed per unit time, the total no. of tasks processed with processing delay D_p. The task scheduling delay D_s with task load L and available processing resources R and the sensitivity factor for priority-based processing α. The priority level of the task P and overall computational efficiency was analyzed for the task load i, having the computational capability of the assigned unit C_i, in which the system sensitivity reached downtime β, and the total no. of tasks N. For evaluating dynamic resource allocation U with the allocation for the task R_i which has a priority weight of task i, and total available resources R_{total} helps to optimize processing time under system constraints. This model ensures optimal task execution, efficient resource allocation, and minimal downtime under dynamic load conditions in multi-agent industrial systems, as illustrated in Figures 13a and 13b. Tables 8a, 8b and 8c present the comparative analysis summary for tasks and machines.

Table 8a. Comparative Analysis Summary for Tasks (Results are mean ± 95% CI from 30 runs; significance vs. ETM-TRDF: $p < 0.01$ for all IIAF metrics)

Metrics	GDPC	COF-GRE	ETM-TRDF	IIAF
Allocation Rate	0.77	0.806	0.85	0.879
Information Lag	0.112	0.097	0.076	0.0547
Allocation Time (s)	6.91	5.92	4.23	2.12
Accuracy Ratio	85.56	89.38	90.64	92.456
Overloaded Tasks	94	74	50	32
Downtime (hours)	8.5	7.9	6.8	5.3

To strengthen the reliability of the reported improvements, each comparative experiment was repeated over 30 independent simulation runs using different random seeds. For every metric, mean values with 95% confidence intervals were

computed, and a two-tailed paired t-test was conducted against the strongest baseline method (ETM-TRDF) at the significance level $\alpha = 0.05$. Across all evaluated metrics, the proposed IIAF framework achieved statistically significant improvements with $p < 0.01$, confirming that the observed gains are reproducible.
To benchmark IIAF against the latest advances, we compare it with three recent state-of-the-art models from the manuscript's reference list that employ deep learning or reinforcement learning:

- **Rajawat et al. (2023)** [8] – A reinforcement-learning-based cognitive adaptive system for industrial IoT.
- **Alkato & Kalenen (2025)** [2] – A transformer-based learning model for dynamic pattern analysis.
- **Khakifirooz et al. (2026)** [7] – An AI-driven scheduling framework integrating theory of constraints.

Table 8b. Performance vs. recent (2023–2025) DRL baselines (mean ± 95% CI, 30 runs)

Metric	IIAF (proposed)	Rajawat et al. [8] (2023)	Alkato & Kalenen [2] (2025)	Khakifirooz et al. [7] (2026)
Allocation rate	0.879 ± 0.008	0.844 ± 0.010	0.851 ± 0.009	0.860 ± 0.008
Information lag (ms)	0.0547 ± 0.001	0.0635 ± 0.002	0.0602 ± 0.002	0.0581 ± 0.001
Allocation time (s)	2.12 ± 0.05	3.05 ± 0.08	2.91 ± 0.07	2.73 ± 0.06
Accuracy (%)	92.46 ± 0.5	89.37 ± 0.7	90.15 ± 0.6	91.02 ± 0.5
Overloaded tasks	32 ± 2	42 ± 3	39 ± 3	36 ± 2
Downtime (h)	5.3 ± 0.1	6.4 ± 0.2	6.1 ± 0.2	5.8 ± 0.1

Table 8b summarizes the results. IIAF significantly outperforms all three recent baselines across every metric (paired t-test, $p < 0.01$). Compared to the best baseline (Khakifirooz et al.), IIAF improves allocation rate by **+2.2%**, reduces information lag by **–5.8%**, cuts allocation time by **–22.3%**, and lowers downtime by **–8.6%**. This demonstrates that IIAF's confidence-gated pre-classification and partial learning constitute a genuine advance beyond current deep reinforcement learning schedulers.

Cross-validation has been added using two public real-world datasets (NIST Smart Manufacturing [25] and Kaggle Predictive Maintenance) together with three task-arrival profiles (λ = 2, 5, 8 tasks/s) given in Table 8c.

Table 8c. Multi-source validation of IIAF

Validation source	Configuration	Accuracy / Metric	Result
OPNET simulation (30 runs)	800 tasks, 60 machines	Allocation accuracy	92.5% ± 0.8%
NIST real dataset [25]	15,000 tasks, 30 CNC machines		91.2%
UCI Predictive Maintenance [36]	10,000 samples, 1 machine		90.7%

Cross-domain low load	λ = 2 tasks/s	Info lag	0.035 ms
Cross-domain nominal load	λ = 5 tasks/s		0.054 ms
Cross-domain high load	λ = 8 tasks/s		0.072 ms

The maximum deviation between the primary OPNET simulation given in Table 8c and the two real-world datasets is only 1.8% . A paired t-test shows this difference is not statistically significant (p = 0.18). The three cross-domain profiles further confirm that IIAF maintains consistent performance across a wide range of task arrival intensities. Thus, the framework is not validated exclusively by simulation; real-world evidence is provided. Table 9 details the comparative analysis summary for machines against comparative baselines and IIAF.

Table 9. Comparative Analysis Summary for Machines

Metrics	**GDPC**	**COF-GRE**	**ETM-TRDF**	**IIAF**
Allocation Rate	0.838	0.893	0.92	0.992
Information Lag	0.11	0.069	0.079	0.057
Allocation Time (s)	6.93	4.87	4.15	2.214
Accuracy Ratio	90.37	92.16	95.28	97.879
Overloaded Tasks	69	30	13	3
Downtime (hours)	7.2	6.5	5.9	4.3

The percentage improvements were calculated directly from the raw comparative values reported in Table 10 using standard relative-performance analysis. For benefit-oriented metrics such as allocation rate and accuracy, the improvement was computed as $(P_{IIAF} - P_{Baseline})/P_{Baseline} \times 100$. For cost-oriented metrics such as information lag, overloaded tasks, and downtime, the reduction was computed as $(P_{Baseline} - P_{IIAF})/P_{Baseline} \times 100$. In the revised manuscript, all percentage claims have been recalculated against the strongest baseline method (ETM-TRDF) to ensure consistency, transparency, and rigorous benchmarking. Accordingly, the proposed IIAF improves allocation rate by 3.41%, accuracy ratio by 2.00%, while reducing information lag by 28.03%, overloaded tasks by 36.00%, and downtime by 22.06% compared with ETM-TRDF.

Table 10. Statistical validation of IIAF against baselines (mean ± 95% CI, n = 30)

Metric	**IIAF (proposed)**	**GDPC [30]**	**COF-GRE [27]**	**ETM-TRDF [29]**	**p-value (IIAF vs ETM-TRDF)**	**Improvement**
Allocation rate	0.879 ± 0.008	0.77 ± 0.02	0.806 ± 0.01	0.85 ± 0.01	< 0.001	+7.03%
Information lag (ms)	0.0547 ± 0.001	0.112 ± 0.004	0.097 ± 0.003	0.076 ± 0.002	< 0.001	−12.03%
Allocation time (s)	2.12 ± 0.05	6.91 ± 0.15	5.92 ± 0.12	4.23 ± 0.08	< 0.001	−20.89%

Accuracy (%)	92.46 ± 0.5	85.56 ± 1.2	89.38 ± 0.9	90.64 ± 0.7	0.003	+11.78%
Overloaded tasks	32 ± 2	94 ± 5	74 ± 4	50 ± 3	< 0.001	−18.65%
Downtime (hours)	5.3 ± 0.1	8.5 ± 0.3	7.9 ± 0.2	6.8 ± 0.15	< 0.001	−37.4%

Table 10 compares IIAF with GDPC [30], COF-GRE [27], and ETM-TRDF [29] across six metrics (mean ± 95% CI, 30 runs). Paired *t*-tests (α = 0.05) against ETM-TRDF show all improvements are significant ($p < 0.01$). IIAF achieves 0.879 ± 0.008 allocation rate vs 0.85 ± 0.01 (+7.03%), with additional gains in information lag (−12.03%), allocation time (−20.89%), accuracy (+11.78%), overloaded tasks (−18.65%), and downtime (−37.4%). These results, along with extended comparisons to recent DRL baselines [2,7,8], confirm consistent and statistically robust performance advantages. Statistical validation was performed to ensure robustness and reproducibility (Table 11). Each experiment was repeated over 30 independent runs (seeds 42–71). Results are reported as mean ± 95% confidence intervals (Student's *t*, df = 29). Statistical significance was assessed using a two-tailed paired *t*-test ($\alpha = 0.05$) against ETM-TRDF, with all improvements significant ($p < 0.01$).

Table 11. Statistical validation settings and significance testing parameters for repeated simulation experiments.

Parameter	Value
Independent runs	30
Random seeds	42–71
Confidence level	95%
Confidence method	Student's t-distribution
Degrees of freedom	29
Statistical test	Two-tailed paired t-test
Significance level	α = 0.05
Baseline comparator	ETM-TRDF
Reported significance	p < 0.01

4.7 *Ablation Study*

To quantify the independent contribution of each IIAF architectural component, an ablation study was conducted under the 800-task, 60-machine configuration across 30 independent simulation runs (random seeds 42–71). Table 12 reports mean ± standard deviation for allocation rate, classification accuracy, and overloaded tasks when each module is systematically disabled while retaining all others. Statistical significance was assessed using paired two-tailed t-tests with Bonferroni correction for multiple comparisons (adjusted $\alpha = 0.0125$).

Table 12. Ablation study results (mean ± SD, n = 30 runs)

Component removed	Allocation rate	Accuracy (%)	Overloaded tasks
None (full IIAF)	0.879 ± 0.008	92.46 ± 0.52	32 ± 2.1
Pre-classification	0.801 ± 0.012 (−8.9%)	87.23 ± 0.78 (−5.7%)	51 ± 3.4 (+19)

Partial learning	0.842 ± 0.010 (−4.2%)	89.16 ± 0.65 (−3.3%)	43 ± 2.7 (+11)
Cognitive decision	0.861 ± 0.009 (−2.0%)	91.02 ± 0.58 (−1.4%)	39 ± 2.4 (+7)
Consistent allocation	0.870 ± 0.008 (−1.0%)	91.85 ± 0.54 (−0.6%)	36 ± 2.2 (+4)

All degradations were statistically significant versus full IIAF (paired *t*-test, Bonferroni-corrected): $p < 0.001$for pre-classification and partial learning removal, and $p < 0.01$for cognitive decision and consistent allocation. Pre-classification caused the largest degradation (−8.9% allocation rate, −5.7% accuracy, +19 overloaded tasks), followed by partial learning (−3.3% accuracy, +11 tasks). Cognitive decision improved allocation by +2.0% and reduced overload by 7 tasks, while consistent allocation reduced overload by 4 and ensured stability.

4.8 *Systematic Sensitivity Analysis*

A systematic sensitivity analysis (Table 13) evaluates robustness by applying ±20% perturbations to six parameters: O_{max}, Z_{thresh}, θ, λ, α_{lr}, and batch size. The impact on Allocation Rate (AR) and Information Lag (IL) remains bounded (≤4.1% for AR, ≤2.8% for IL), indicating stable performance under variability. AR is most sensitive to O_{max}, where a +20% increase reduces AR by 4.1% due to prolonged machine overload. IL is most sensitive to Z_{thresh}; a −20% reduction decreases IL by 2.8% but increases allocation time by 6.3%. The learning rate α_{lr}shows minimal sensitivity (<1% change in AR), confirming stable convergence. Overall, IIAF demonstrates robustness to realistic variations in system parameters.

Table 13. Sensitivity analysis summary

Parameter	**Change**	**ΔAR (%)**	**ΔIL (%)**
O_{max}	+20%	−4.1	+1.2
O_{max}	−20%	+2.3	−0.8
Z_{thresh}	+20%	+0.5	+1.9
Z_{thresh}	−20%	−1.2	−2.8
θ (conf. threshold)	+20% (0.5→0.6)	−0.3	−0.5
θ	−20% (0.5→0.4)	−2.3	−1.1
λ (failure rate)	+20%	−1.8	+2.1
α_{lr} (learning rate)	±20%	<1.0	<0.5
Batch size	±20%	<0.8	<0.6

The confidence threshold θ demonstrates a non-linear effect: reducing $\theta$from 0.5 to 0.4 increases autonomous decisions by 18%, but introduces a 2.3% reduction in classification accuracy. Overall, these findings indicate that the default parameter configuration represents a stable operating point and that IIAF maintains reliable performance under realistic parameter variability. Hence, the adopted conditional decision rules and weighted scoring mechanisms do not introduce brittleness into the IIAF proposed framework.

4.9 *Real-Time Optimization Specification*

The IIAF optimization process runs at a base frequency of 0.2 Hz (every 5

seconds) under normal conditions. When the system enters a congested state (trigger: information lag $L > 0.08$ ms or overload ratio $O > 0.85$), the frequency adaptively increases to 1 Hz (every 1 second). Over 30 independent runs on a 60-machine configuration, the mean processing time per full optimization cycle (including cognitive detection, pre-classification, and allocation) is 2.12 ± 0.31 ms. The worst-case latency, which includes classifier update and decision logging, is 47 ms. Table 14 summarizes these performance metrics.

Table 14. Real-time optimization performance (measured over 30 runs)

Operating State	Optimization Frequency	Processing Time (mean ± SD)	Worst-Case Latency
Normal	0.2 Hz (every 5 s)	1.85 ± 0.22 ms	38 ms
Congested	1 Hz (every 1 s)	2.12 ± 0.31 ms	47 ms

5 Conclusion

This paper develops the IIAF, or the Intelligent Industrial Analytical Framework, which attempts to manage cognitive-aware task and resource problems more effectively in sophisticated industrial settings. Incorporating features such as pre-classification, dynamic task assignment, and live machine observation, the framework provides a solution to the foremost inefficiencies of a traditional approach to task allocation. The efficiency of the system, as implemented using OPNET simulations, validated the accuracy of the proposed framework. The IIAF achieved up to 7.03% higher allocation rates, 11.78% greater task classification accuracy, and a 12.03% decrease in information lag compared to the baseline models. Additionally, it demonstrated a 20.89% reduction in allocation time, an 18.65% decrease in overloaded tasks, and a 37.4% decrease in system downtime. Under a secondary test set, the model demonstrated sustained improvements, with a 10.83% increase in allocation rate, a 10.55% increase in accuracy, and an 8.7% decrease in information lag. Furthermore, the model validated the robustness of the framework, achieving a 19.44% increase in speed for allocation, a 15.34% reduction in overloads, and a 40.7% reduction in downtime, thereby proving the adaptability of the framework across diverse industrial conditions. The scope of this work is limited to structured task-allocation data; unstructured data processing is not addressed and remains outside the current framework's capabilities.

IIAF introduces three implementation limitations with defined mitigation paths: (i) edge latency due to the pre-classifier (12 inputs, 64–32 neurons), to be reduced via a distilled model (16–8 neurons) achieving 89.1% accuracy at 23% inference cost; (ii) reliance on simulated data, to be addressed through adversarial domain adaptation with industrial deployment partners; and (iii) absence of adversarial robustness, to be mitigated using a lightweight PBFT-based Byzantine fault-tolerant consensus layer for secure allocation.

Acknowledgment

Data availability

The NIST Smart Manufacturing dataset [25] is publicly available. The UCI Predictive Maintenance dataset (Matzka, 2020) is available at [36]. The source code, OPNET simulation files, task generation scripts, and hyperparameter tuning logs are deposited in a public GitHub repository and will be released upon acceptance. All other data generated during this study are available from the corresponding author upon reasonable request.

Conflict of interest

The authors declare that they have no conflict of interest.

Funding

This work was financially supported by the Science and Technology Research Project of Chongqing Education Commission (KJQN202503006).

References

1. Mourtzis, D. & Sgarbossa, F. Operations management and information systems for smart manufacturing. In Manufacturing from Industry 4.0 to Industry 5.0, 267–288 (Elsevier, 2024).
2. Alkato, M. A., & Kalenen, N. Dynamic pattern analysis for enhanced predictive intelligence in smart environments using transformer learning models. PatternIQ Mining, 2(1), 85–96. (2025)
3. Sharma, V., Khang, A., Hiwarkar, P. & Jadhav, B. Robotic process automation applications in data management. In AI-centric Modeling and Analytics, 238–259 (CRC Press, 2023).
4. Valaskova, K., Nagy, M. & Grecu, G. Digital twin simulation modeling, artificial intelligence-based Internet of Manufacturing Things systems, and virtual machine and cognitive computing algorithms in the Industry 4.0-based Slovak labor market. Oeconomia Copernicana 15, 95–143 (2024).
5. Wang, Z., Wang, L., Yuan, Z. & Chen, B. Data-driven optimal operation of the industrial methanol to olefin process based on relevance vector machine. Chin. J. Chem. Eng. 28, 1–10 (2020).
6. Girija, R., Vedhapriyavadhana, R., Jayalakshmi, S. L. & Ahmed, H. F. T. Smart AI: adapting to sensor data in practical applications. In *Adaptive AI in Sensor Informatics*, 203–227 (Elsevier, 2026).
7. Khakifirooz, M., Fathi, M. & Dolgui, A. Theory of AI-driven scheduling (TAIS): a service-oriented scheduling framework by integrating theory of constraints and AI. *Int. J. Prod. Res*. 64, 642–676 (2026).
8. Rajawat, A. S. et al. Cognitive adaptive systems for industrial internet of things using reinforcement algorithm. Electronics 12, 217 (2023).
9. Rehman, M. H. et al. The role of big data analytics in industrial Internet of Things. Futur. Gener. Comput. Syst. 99, 247–259 (2019).
10. Kabugo, J. C., Jämsä-Jounela, S. L., Schiemann, R. & Binder, C. Industry 4.0-based process data analytics platform: A waste-to-energy plant case study. Int. J. Electr. Power Energy Syst. 115, 105508 (2020).
11. Yu, W. & Zhao, C. Low-rank characteristic and temporal correlation analytics for incipient industrial fault detection with missing data. IEEE Trans. Ind. Inform. 16, 1–12 (2020).
12. Syed, D. et al. A comparative analysis of metaheuristic techniques for high availability systems. IEEE Access 12, 7382–7398 (2024).
13. Xu, W. et al. Accelerating federated learning for IoT in big data analytics with pruning, quantization and selective updating. IEEE Access 9, 38457–38466 (2021).
14. Liu, X. et al. Reinforcement learning-based multislot double-threshold spectrum sensing with Bayesian fusion for industrial big spectrum data. IEEE Trans. Ind.

Inform. 17, 3391–3400 (2020).

15. Shah, D., Wang, J. & He, Q. P. Feature engineering in big data analytics for IoT-enabled smart manufacturing–comparison between deep learning and statistical learning. Comput. Chem. Eng. 141, 106970 (2020).
16. Karagiorgos, A., Lazos, G., Stavropoulos, A., Karagiorgou, D. & Valkani, F. Information system assisted knowledge accounting and cognitive managerial implications. EuroMed J. Bus. 20, 1–20 (2025).
17. Kumar, A. & Jaiswal, A. A deep swarm-optimized model for leveraging industrial data analytics in cognitive manufacturing. IEEE Trans. Ind. Inform. 17, 2938–2946 (2020).
18. Algarni, F. A novel quality-based computation offloading framework for edge cloud-supported Internet of Things. Alexandria Eng. J. 70, 585–599 (2023).
19. Almusawi, A. & Pugazhenthi, S. Innovative resource distribution through multi-agent supply chain scheduling leveraging honey bee optimization techniques. PatternIQ Mining 1, 48–62 (2024).
20. Yuan, X., Qi, S., Wang, Y. & Xia, H. A dynamic CNN for nonlinear dynamic feature learning in soft sensor modeling of industrial process data. Control Eng. Pract. 104, 104614 (2020).
21. Qu, X., Li, Z. & Wang, H. A high-concurrency tasks emergency offloading framework for industrial edge networks. IETE J. Res. 1, 1–15 (2024).
22. Zhang, T. et al. ihppvis: Interactive visual analysis of industrial data in heavy plate production. IFAC-PapersOnLine 53, 12050–12055 (2020).
23. Subramani, S. S., Shakeel, P. M., Bin Mohd Aboobaider, B. & Salahuddin, L. B. Classification learning assisted biosensor data analysis for preemptive plant disease detection. ACM Trans. Sens. Netw. 18, 1–19 (2022).
24. Zhang, T. et al. Industrial text analytics for reliability with derivative-free optimization. Comput. Chem. Eng. 135, 106763 (2020).
25. Zhang, X., Ming, X. & Yin, D. Reference architecture of common service platform for Industrial Big Data (I-BD) based on multi-party co-construction. Int. J. Adv. Manuf. Technol. 105, 1949–1965 (2019).
26. Helu, M., Sprock, T., Hartenstine, D., Venketesh, R. & Sobel, W. Scalable data pipeline architecture to support the industrial Internet of Things. CIRP Ann. 69, 385–388 (2020).
27. Luo, L., Bao, S. & Peng, X. Robust monitoring of industrial processes using process data with outliers and missing values. Chemom. Intell. Lab. Syst. 192, 103827 (2019).
28. Zangeneh, P. & McCabe, B. Ontology-based knowledge representation for industrial megaprojects analytics using linked data and the semantic web. Adv. Eng. Inform. 46, 101164 (2020).
29. Zhang, X. & Ge, Z. Automatic deep extraction of robust dynamic features for industrial big data modeling and soft sensor application. IEEE Trans. Ind. Inform. 16, 4456–4467 (2019).
30. Diaz-Rozo, J., Bielza, C. & Larrañaga, P. Clustering of data streams with dynamic Gaussian mixture models: an IoT application in industrial processes. IEEE

Internet Things J. 5, 3533–3547 (2018).

31. Cirera, J., Carino, J. A., Zurita, D. & Ortega, J. A. Data analytics for performance evaluation under uncertainties applied to an industrial refrigeration plant. IEEE Access 7, 64127–64135 (2019).
32. Ragazou, K., Passas, I., Garefalakis, A., Galariotis, E. & Zopounidis, C. Big data analytics applications in information management driving operational efficiencies and decision-making: Mapping the field of knowledge with bibliometric analysis using R. Big Data Cogn. Comput. 7, 13 (2023).
33. Renna, P., Materi, S. & Ambrico, M. Review of responsiveness and sustainable concepts in cellular manufacturing systems. *Appl. Sci.* 13, 1125 (2023)
34. Prashar, A., Tortorella, G. L. & Fogliatto, F. S. Production scheduling in Industry 4.0: Morphological analysis of the literature and future research agenda. *J. Manuf. Syst.* 65, 33–43 (2022).
35. Tliba, K., Diallo, T. M., Penas, O., Ben Khalifa, R., Ben Yahia, N. & Choley, J. Y. Digital twin-driven dynamic scheduling of a hybrid flow shop. *J. Intell. Manuf.* 34, 2281–2306 (2023).
36. UCI Machine Learning Repository Matzka, S. AI4I 2020 Predictive Maintenance Dataset (Version 1) [Dataset]. *UCI Machine Learning Repository*. https://doi.org/10.24432/C5HS5C (2020).